

Mathematical Notes: Rounded-Polygon Packing Model

Derivations of all algorithms in pyCudaPolygon

May 16, 2026

Contents

1	Overview	3
2	Polygon Geometry	3
2.1	Backbone polygon	3
2.1.1	Edge vectors	3
2.1.2	Area (shoelace formula)	4
2.1.3	Equilateral constraint	4
2.2	Shape index	4
2.3	Exterior angle at a vertex	4
3	Rounded-Polygon Boundary	4
3.1	Inward arc rounding	4
3.2	Tangent points and arc centre	5
3.3	Straight edge section	6
3.4	Area of rounded polygon	7
4	Intersection Area as the Energy	7
4.1	Setup	7
4.2	Green's theorem	7
4.3	Contribution from a straight edge segment	8
4.4	Contribution from an arc segment	9
4.5	Point-in-smoothed-polygon test	9
4.6	Finding intersections	10
4.6.1	Arc-edge intersection (<code>isectArcEdge</code>)	10
4.6.2	Arc-arc intersection (<code>isectArcArc</code>)	10
4.6.3	Edge-edge intersection (<code>isectEdgeEdge</code>)	10

5	Force as Gradient of Intersection Area	10
5.1	General Leibniz rule for signed area	10
5.2	Gradient for a straight edge segment	11
5.3	Gradient for an arc segment	12
5.4	Arc geometry chain rule (<code>arcGeomGrad</code>)	12
6	SHAKE Constraint Projection	13
6.1	Constraints	13
6.2	Newton step	13
6.3	Jacobian structure	13
6.4	Gram matrix JJ^T	14
6.5	LU solve and position update	14
6.6	Convergence	14
7	Force Projection	14
7.1	Goal	14
7.2	Edge gradient matrix and SVD	15
7.3	Area gradient orthogonalisation (<code>gramSchmidtAreaKernel</code>)	15
7.4	Full force projection (<code>forceProjectFullKernel</code>)	15
8	FIRE Minimization	16
8.1	Background	16
8.2	Velocity-Verlet integration	16
8.3	FIRE steering	16
8.4	Time-step scaling for dense packings	17
9	Shape Generation via SLSQP	17
9.1	Goal	17
9.2	Angle parameterisation	17
9.3	Closure constraint	18
9.4	Shape index constraint	18
9.5	SLSQP formulation	18
10	Bidisperse Packing Setup	19
10.1	Target areas	19
10.2	Target edge lengths	19
10.3	Rounding radius	19
10.4	Packing fraction for the 2-polygon test	19
11	Periodic Boundary Conditions	20
12	Summary of Computational Flow	20

1 Overview

This document gives step-by-step derivations of every piece of mathematics used in the pyCudaPolygon CUDA code for soft rounded-polygon packing. The chain is:

1. **Polygon geometry**: equilateral closed polygons, shape index.
2. **Rounded-polygon boundary**: inward arc rounding at each vertex.
3. **Arc geometry** (`computeArcGeom`): tangent points, arc centre, angle span.
4. **Intersection area as energy**: Green’s theorem applied to arcs and straight edges.
5. **Force as gradient of intersection area**: analytic gradient via the arc-geometry chain rule (`arcGeomGrad`).
6. **SHAKE**: simultaneous Newton-step constraint projection for equilateral edge lengths and area.
7. **Force projection**: SVD + Gram–Schmidt to remove all constraint-space force components.
8. **FIRE minimization**: velocity-Verlet FIRE with SHAKE rederivation.
9. **Shape generation**: SLSQP for random equilateral polygons with prescribed shape index.
10. **Bidisperse packing setup**: area formulas from packing fraction and perimeter ratio.

Throughout, positions live in a periodic $[0, 1)^2$ box with minimum-image convention

$$\text{wrap}(x) = x - \lfloor x + \tfrac{1}{2} \rfloor.$$

2 Polygon Geometry

2.1 Backbone polygon

A polygon P with N vertices is represented by its backbone vertices $\mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_{N-1} \in [0, 1)^2$ (periodic box), traversed counterclockwise. Edges are labelled E_k , running from $\mathbf{a}_k$ to $\mathbf{a}_{k+1}$ (indices mod N).

2.1.1 Edge vectors

The (wrapped) edge vectors are

$$\mathbf{u}_k = \text{wrap}(\mathbf{a}_k - \mathbf{a}_{k-1}), \quad \mathbf{v}_k = \text{wrap}(\mathbf{a}_{k+1} - \mathbf{a}_k).$$

Unit vectors: $\hat{\mathbf{u}}_k = \mathbf{u}_k / |\mathbf{u}_k|$, $\hat{\mathbf{v}}_k = \mathbf{v}_k / |\mathbf{v}_k|$.

2.1.2 Area (shoelace formula)

The signed area of the backbone polygon (positive for CCW) is

$$A_{\text{bb}} = \frac{1}{2} \sum_{k=0}^{N-1} (x_k y_{k+1} - x_{k+1} y_k),$$

where $(x_k, y_k) = \mathbf{a}_k$ and all coordinates are interpreted in a single minimum-image frame anchored at $\mathbf{a}_0$.

2.1.3 Equilateral constraint

SHAKE enforces $|\mathbf{v}_k| = L_k = \text{constant}$ for all k . When all $L_k = L$ the polygon is *equilateral* (equal edge lengths but not necessarily equiangular).

2.2 Shape index

For a polygon with perimeter $P = \sum_k L_k$ and backbone area A , the *shape index* (also called the reduced perimeter) is

$$\kappa = \frac{P}{\sqrt{A}}.$$

A circle has $\kappa = 2\sqrt{\pi} \approx 3.545$; a square has $\kappa = 4$; elongated or irregular shapes have larger κ . In the bidisperse packing, we use $\kappa = 3.7$ for both polygon sizes.

2.3 Exterior angle at a vertex

At vertex $\mathbf{a}_k$ the incoming direction is $\hat{\mathbf{u}}_k$ and the outgoing direction is $\hat{\mathbf{v}}_k$. Define

$$c_k = \hat{\mathbf{u}}_k \times \hat{\mathbf{v}}_k = (\hat{u}_x)(\hat{v}_y) - (\hat{u}_y)(\hat{v}_x), \quad d_k = \hat{\mathbf{u}}_k \cdot \hat{\mathbf{v}}_k = \hat{u}_x \hat{v}_x + \hat{u}_y \hat{v}_y.$$

The *exterior angle* (the angle you turn through at vertex k , positive for a left turn in a CCW polygon) is

$$\varphi_k = \text{atan2}(c_k, d_k).$$

For a convex polygon traversed CCW, $\varphi_k > 0$ for all k and $\sum_k \varphi_k = 2\pi$.

3 Rounded-Polygon Boundary

3.1 Inward arc rounding

At each vertex $\mathbf{a}_k$ we replace the sharp corner with a circular arc of radius $\delta > 0$. The arc is tangent to both adjacent (original) edges and curves *inward* (toward the polygon interior). The resulting boundary alternates between straight edge sections and smooth arcs:

$$\partial \tilde{P} = K_0, E_0, K_1, E_1, \dots, K_{N-1}, E_{N-1},$$

where K_k is the arc at vertex $\mathbf{a}_k$ and E_k is the straight section of edge k between adjacent tangent points.

This is **not** the Minkowski sum $P \oplus D(\delta)$ (which rounds outward); it is instead an *inscribed* rounding that cuts each corner.

3.2 Tangent points and arc centre

The following derivation corresponds exactly to `computeArcGeom` in `kernels.cuh`.

Tangent set-back ℓ_k . The arc at $\mathbf{a}_k$ must be tangent to the incoming edge (direction $\hat{\mathbf{u}}_k$) and to the outgoing edge (direction $\hat{\mathbf{v}}_k$). Let the tangent point on the incoming edge be $\mathbf{a}_k^- = \mathbf{a}_k - \ell_k \hat{\mathbf{u}}_k$ and on the outgoing edge be $\mathbf{a}_k^+ = \mathbf{a}_k + \ell_k \hat{\mathbf{v}}_k$.

For a circle of radius δ centred at $\mathbf{z}_k$ to be tangent to the incoming edge at $\mathbf{a}_k^-$, we need:

$$(\mathbf{z}_k - \mathbf{a}_k^-) \perp \hat{\mathbf{u}}_k \quad \text{and} \quad |\mathbf{z}_k - \mathbf{a}_k^-| = \delta.$$

By symmetry (same tangency condition on outgoing edge), $|\mathbf{z}_k - \mathbf{a}_k^+| = \delta$ as well. The tangent-setback formula follows from the two perpendicularity conditions:

Write $\mathbf{z}_k = \mathbf{a}_k + \alpha \hat{\mathbf{u}}_k + \beta \hat{\mathbf{n}}_k$ where $\hat{\mathbf{n}}_k$ is a unit vector perpendicular to $\hat{\mathbf{u}}_k$. For the arc to be tangent to the incoming edge: the component of $(\mathbf{z}_k - \mathbf{a}_k^-)$ along $\hat{\mathbf{u}}_k$ must vanish:

$$\alpha + \ell_k = 0 \implies \alpha = -\ell_k.$$

For the arc to be tangent to the outgoing edge (direction $\hat{\mathbf{v}}_k$): the component of $(\mathbf{z}_k - \mathbf{a}_k^+)$ along $\hat{\mathbf{v}}_k$ must vanish. Writing $\hat{\mathbf{v}}_k = d_k \hat{\mathbf{u}}_k + c_k \hat{\mathbf{n}}_k$ (resolved in the $\hat{\mathbf{u}}_k, \hat{\mathbf{n}}_k$ frame):

$$(-\ell_k - \ell_k d_k) + c_k \beta = 0 \implies \beta = \frac{\ell_k(1 + d_k)}{c_k}.$$

Using $|\mathbf{z}_k - \mathbf{a}_k^-| = \delta$:

$$\alpha^2 + \beta^2 - 2\alpha\ell_k + \ell_k^2 = \delta^2 \implies \ell_k^2 + \beta^2 = \delta^2.$$

Since $\beta = \ell_k(1 + d_k)/c_k$ and $\beta^2 = \delta^2 - \ell_k^2$:

$$\ell_k^2 \left[1 + \frac{(1 + d_k)^2}{c_k^2} \right] = \delta^2 \implies \ell_k^2 \cdot \frac{c_k^2 + (1 + d_k)^2}{c_k^2} = \delta^2.$$

Expanding $c_k^2 + (1 + d_k)^2 = 1 - d_k^2 + 1 + 2d_k + d_k^2 = 2(1 + d_k)$:

$$\ell_k^2 \cdot \frac{2(1 + d_k)}{c_k^2} = \delta^2 \implies \boxed{\ell_k = \frac{\delta |c_k|}{1 + d_k} = \delta \tan\left(\frac{\varphi_k}{2}\right)}.$$

The last equality uses the half-angle identity $\tan(\varphi/2) = \sin \varphi / (1 + \cos \varphi)$ together with $|c_k| = \sin \varphi_k$, $d_k = \cos \varphi_k$.

Validity condition. For the tangent points to lie within the edges (so adjacent arcs do not overlap), we need $\ell_k + \ell_{k-1} < L_{k-1}$ for every edge $k-1$. Since SHAKE maintains equilateral edges, a sufficient condition is

$$2\delta \tan\left(\frac{\varphi_k}{2}\right) < L \quad \text{for all } k.$$

For a regular N -gon this gives $\delta < L/(2 \tan(\pi/N))$.

Arc centre. From $\alpha = -\ell_k$ and $\beta = \ell_k(1 + d_k)/c_k = \delta|c_k|/c_k = \delta \operatorname{sgn}(c_k)$:

$$\mathbf{z}_k = \mathbf{a}_k - \ell_k \hat{\mathbf{u}}_k + \delta \operatorname{sgn}(c_k) \hat{\mathbf{n}}_k.$$

Using $\hat{\mathbf{n}}_k = (-\hat{u}_y, \hat{u}_x)$ (the 90° CCW rotation of $\hat{\mathbf{u}}_k$) and noting that for a CCW polygon $c_k > 0$, this simplifies to $\beta = \delta$, i.e.:

$$\mathbf{z}_k = \mathbf{a}_k - \ell_k \hat{\mathbf{u}}_k + \delta \hat{\mathbf{n}}_k.$$

This can be rewritten as

$$\boxed{\mathbf{z}_k = \mathbf{a}_k + \frac{\delta(\hat{\mathbf{v}}_k - \hat{\mathbf{u}}_k)}{|c_k|},}$$

which is what the code computes. *Proof:* $-\ell_k \hat{\mathbf{u}}_k + \delta \hat{\mathbf{n}}_k = -\frac{\delta|c_k|}{1+d_k} \hat{\mathbf{u}}_k + \delta(-\hat{u}_y, \hat{u}_x)$. Meanwhile $(\hat{\mathbf{v}}_k - \hat{\mathbf{u}}_k)/|c_k|$: write $\hat{\mathbf{v}}_k - \hat{\mathbf{u}}_k = (d_k - 1)\hat{\mathbf{u}}_k + c_k \hat{\mathbf{n}}_k$, so $(\hat{\mathbf{v}}_k - \hat{\mathbf{u}}_k)/|c_k| = (d_k - 1)\hat{\mathbf{u}}_k/|c_k| + \operatorname{sgn}(c_k)\hat{\mathbf{n}}_k$. Multiplying by δ : $\delta(d_k - 1)/|c_k| \cdot \hat{\mathbf{u}}_k + \delta \operatorname{sgn}(c_k)\hat{\mathbf{n}}_k$. Since $(d_k - 1)/|c_k| = -(1 - d_k)/(|c_k|) = -(1 + d_k - 2)/(|c_k|) \dots$ a cleaner route: with $\varphi = \varphi_k$, $c = \sin \varphi$, $d = \cos \varphi$:

$$\begin{aligned} \frac{\delta(\hat{\mathbf{v}} - \hat{\mathbf{u}})}{|c|} &= \frac{\delta}{|\sin \varphi|} [(\cos \varphi - 1)\hat{\mathbf{u}} + \sin \varphi \hat{\mathbf{n}}] \\ &= \delta \frac{\cos \varphi - 1}{\sin \varphi} \hat{\mathbf{u}} + \delta \hat{\mathbf{n}} = -\delta \tan \frac{\varphi}{2} \hat{\mathbf{u}} + \delta \hat{\mathbf{n}} = -\ell_k \hat{\mathbf{u}}_k + \delta \hat{\mathbf{n}}_k. \end{aligned}$$

Arc angle span. The arc runs from $\mathbf{a}_k^-$ to $\mathbf{a}_k^+$ centred at $\mathbf{z}_k$. The starting angle (angle of $\mathbf{a}_k^- - \mathbf{z}_k$ measured from the positive x -axis) is

$$\phi_{0,k} = \operatorname{atan2}(a_{k,y}^- - z_{k,y}, a_{k,x}^- - z_{k,x}).$$

The arc sweeps through $\Delta\varphi_k = \varphi_k$ (the exterior angle), i.e. the arc spans exactly the same angle as the turn at the vertex.

3.3 Straight edge section

The edge section E_k (the flat part of edge k) runs from $\mathbf{a}_k^+$ to $\mathbf{a}_{k+1}^-$, a straight line segment of length $L_k - \ell_k - \ell_{k+1}$.

3.4 Area of rounded polygon

The area of the inscribed rounded polygon $\tilde{P}$ is

$$A_{\tilde{P}} = A_{\text{bb}} - \sum_{k=0}^{N-1} \Delta A_k,$$

where ΔA_k is the area of the corner region removed at vertex k . Each removed corner is the region bounded by the arc K_k and the two straight segments from $\mathbf{a}_k$ to $\mathbf{a}_k^\pm$. Its area is

$$\Delta A_k = \frac{1}{2} \ell_k^2 \tan\left(\frac{\varphi_k}{2}\right) - \frac{1}{2} \delta^2 (\varphi_k - \sin \varphi_k).$$

Derivation: The two triangles from $\mathbf{a}_k$ to the tangent points have total area $\frac{1}{2} \ell_k^2 \sin \varphi_k / \cos^2(\varphi_k/2) = \frac{1}{2} \ell_k^2 \tan(\varphi_k/2)$ (using the tangency geometry). The circular sector subtracted is $\frac{1}{2} \delta^2 \varphi_k$ and the triangles from $\mathbf{z}_k$ to $\mathbf{a}_k^\pm$ together contribute $\frac{1}{2} \delta^2 \sin \varphi_k$. The net result is the formula above.

4 Intersection Area as the Energy

4.1 Setup

For two rounded polygons $\tilde{A}$ and $\tilde{B}$ (corresponding to backbone polygons A and B), the potential energy is

$$E_{AB} = \text{Area}(\tilde{A} \cap \tilde{B}).$$

The total energy is $E = \sum_{A < B} E_{AB}$ summed over interacting pairs.

4.2 Green's theorem

Green's theorem (the “shoelace” form) states that for a region Ω with boundary $\partial\Omega$ traversed counterclockwise,

$$\text{Area}(\Omega) = \frac{1}{2} \oint_{\partial\Omega} (x \, dy - y \, dx).$$

We compute $\text{Area}(\tilde{A} \cap \tilde{B})$ by integrating over the boundary of $\tilde{A} \cap \tilde{B}$. The intersection boundary consists of alternating arcs and straight segments from $\partial\tilde{A}$ and $\partial\tilde{B}$.

Key decomposition. Rather than assembling the full intersection boundary (which requires polygon clipping), the CUDA kernel decomposes the computation per feature of $\tilde{A}$:

For each feature F of $\partial\tilde{A}$ (either arc K_i or edge E_i):

1. Clip F against $\tilde{B}$: find the sub-intervals where F lies *inside* $\tilde{B}$.
2. For each such sub-interval $[t_0, t_1]$, add the corresponding signed area contribution $\delta\mathcal{A}$ to the energy.

This is valid because, when we apply Green's theorem to $\tilde{A} \cap \tilde{B}$, the only contributions come from the parts of the boundary that lie inside the other shape. Summing over all features of $\tilde{A}$ gives the full intersection area. (The contributions from $\partial\tilde{B}$ inside $\tilde{A}$ are captured when the roles of A and B are reversed; both halves are summed in the final accumulation via `atomicAdd`.)

4.3 Contribution from a straight edge segment

Edge section E_i runs from $\mathbf{a}_i^+ \equiv \mathbf{P}$ to $\mathbf{a}_{i+1}^- \equiv \mathbf{Q}$ parametrically as

$$\gamma(t) = (1-t)\mathbf{P} + t\mathbf{Q}, \quad t \in [0, 1].$$

Suppose the sub-interval $[t_0, t_1]$ lies inside $\tilde{B}$, with endpoints $\mathbf{X}_0 = \gamma(t_0)$ and $\mathbf{X}_1 = \gamma(t_1)$. The Green's theorem integrand over this segment is

$$d\mathcal{A} = \frac{1}{2}(x dy - y dx).$$

On the straight segment:

$$x = (1-t)P_x + tQ_x, \quad dx = (Q_x - P_x) dt,$$

and similarly for y . Therefore:

$$x dy - y dx = [(1-t)P_x + tQ_x](Q_y - P_y) dt - [(1-t)P_y + tQ_y](Q_x - P_x) dt.$$

Integrating from t_0 to t_1 :

$$\delta\mathcal{A}_{\text{edge}} = \frac{1}{2} \int_{t_0}^{t_1} [(1-t)(P_x Q_y - P_y Q_x) + t(Q_x P_y - Q_y P_x) \cdot (-1) + \dots] dt.$$

After simplification, the result depends only on the *endpoint coordinates* $\mathbf{X}_0$ and $\mathbf{X}_1$ of the inside sub-interval:

$$\boxed{\delta\mathcal{A}_{\text{edge}}([\mathbf{X}_0, \mathbf{X}_1]) = \frac{1}{2}(X_{0x} X_{1y} - X_{0y} X_{1x}).}$$

Proof: For a straight segment from $\mathbf{X}_0$ to $\mathbf{X}_1$: $x = (1-s)X_{0x} + sX_{1x}$, $dx = (X_{1x} - X_{0x}) ds$. Then:

$$\begin{aligned} \int_0^1 (x dy - y dx) &= \int_0^1 [(1-s)X_{0x} + sX_{1x}](X_{1y} - X_{0y}) ds \\ &\quad - \int_0^1 [(1-s)X_{0y} + sX_{1y}](X_{1x} - X_{0x}) ds \\ &= \frac{1}{2}(X_{0x} + X_{1x})(X_{1y} - X_{0y}) - \frac{1}{2}(X_{0y} + X_{1y})(X_{1x} - X_{0x}) \\ &= X_{0x}X_{1y} - X_{0y}X_{1x}. \end{aligned}$$

This matches the kernel line: `localEnergy += 0.5*(X0x*X1y - X0y*X1x).`

4.4 Contribution from an arc segment

Arc K_i at vertex $\mathbf{a}_i$ is a circle of radius δ centred at $\mathbf{z}_i$, parametrised by

$$\gamma(\tau) = \mathbf{z}_i + \delta(\cos(\phi_{0,i} + \tau\Delta\varphi_i), \sin(\phi_{0,i} + \tau\Delta\varphi_i)), \quad \tau \in [0, 1].$$

Suppose the inside sub-interval is $[\tau_0, \tau_1]$ with endpoints $\mathbf{X}_0, \mathbf{X}_1$ and spanning angle $\Delta\psi = (\tau_1 - \tau_0)\Delta\varphi_i$.

The Green's integrand on a circle of radius δ centred at $\mathbf{z}$:

$$x dy - y dx = (z_x + \delta \cos \phi)(-\delta \sin \phi d\phi) - (z_y + \delta \sin \phi)(\delta \cos \phi d\phi) = -\delta(z_x \sin \phi + z_y \cos \phi + \delta) d\phi.$$

Integrate from $\phi_0 = \phi_{0,i} + \tau_0\Delta\varphi_i$ to $\phi_1 = \phi_{0,i} + \tau_1\Delta\varphi_i$:

$$\int (x dy - y dx) = \delta[z_x \cos \phi - z_y \sin \phi]_{\phi_0}^{\phi_1} - \delta^2(\phi_1 - \phi_0).$$

Using $\delta \cos \phi_{0/1} = X_{0/1,x} - z_x$ and $\delta \sin \phi_{0/1} = X_{0/1,y} - z_y$:

$$\begin{aligned} \int (x dy - y dx) &= z_x(X_{1x} - z_x) - z_y(X_{1y} - z_y) - z_x(X_{0x} - z_x) + z_y(X_{0y} - z_y) - \delta^2\Delta\psi \\ &= z_x(X_{1x} - X_{0x}) - z_y(X_{1y} - X_{0y}) - \delta^2\Delta\psi. \end{aligned}$$

We can write $z_x X_{1x} - z_x X_{0x} = X_{0x} X_{1y} - X_{0y} X_{1x} + z_x(X_{1x} - X_{0x}) - z_y(X_{1y} - X_{0y}) + \dots$ — it's cleaner to directly compute. The shoelace contribution for the chord plus the circular cap:

$$\text{Area}(\text{arc sector}) = \frac{1}{2}(X_{0x}X_{1y} - X_{0y}X_{1x}) + \frac{1}{2}\delta^2(\Delta\psi - \sin \Delta\psi).$$

Derivation: The signed area swept by the arc from $\mathbf{X}_0$ to $\mathbf{X}_1$ centred at $\mathbf{z}$ can be split into (i) the triangle $\mathbf{0}-\mathbf{X}_0-\mathbf{X}_1$ with area $\frac{1}{2}(X_{0x}X_{1y} - X_{0y}X_{1x})$ and (ii) the circular segment (area enclosed between the chord $\mathbf{X}_0\mathbf{X}_1$ and the arc). The circular segment area is $\frac{1}{2}\delta^2(\Delta\psi - \sin \Delta\psi)$. Combining:

$$\delta \mathcal{A}_{\text{arc}} = \frac{1}{2}(X_{0x}X_{1y} - X_{0y}X_{1x}) + \frac{1}{2}\delta^2(\Delta\psi - \sin \Delta\psi).$$

This matches the kernel lines:

```
localEnergy += 0.5*(X0x*X1y - X0y*X1x)
              + 0.5*delta*delta*(dpsi - sin(dpsi));
```

4.5 Point-in-smoothed-polygon test

To determine whether a midpoint of a feature segment lies inside $\tilde{B}$, the code uses a horizontal ray-casting algorithm (`pipSmoothed`). A horizontal ray from point (p_x, p_y) toward $+x$ is cast, and the number of crossings with $\partial\tilde{B}$ (both arc and straight segments) is counted. An odd count means the point is inside.

For a straight edge from $\mathbf{b}_j^+$ to $\mathbf{b}_{j+1}^-$: standard ray-segment crossing test.

For an arc at vertex j with centre (z_x, z_y) , radius δ : the ray $y = p_y$ intersects the circle at $x = z_x \pm \sqrt{\delta^2 - (p_y - z_y)^2}$ (if the discriminant is non-negative). Each intersection is checked to lie on the arc (i.e., the corresponding angle $\phi = \text{atan2}(p_y - z_y, x - z_x)$ falls in the arc range $[\phi_0, \phi_0 + \Delta\varphi]$) and to satisfy $x > p_x$.

4.6 Finding intersections

4.6.1 Arc–edge intersection (isectArcEdge)

Arc: centre $\mathbf{z}$, radius δ , angles $[\phi_0, \phi_0 + \Delta\varphi]$. Edge: from $\mathbf{p}$ to $\mathbf{q}$, parametrised as $\mathbf{r}(t) = \mathbf{p} + t\mathbf{e}$, $\mathbf{e} = \mathbf{q} - \mathbf{p}$, $t \in (0, 1)$.

Substituting into the circle equation $|\mathbf{r} - \mathbf{z}|^2 = \delta^2$:

$$|\mathbf{e}|^2 t^2 + 2(\mathbf{f} \cdot \mathbf{e})t + |\mathbf{f}|^2 - \delta^2 = 0, \quad \mathbf{f} = \mathbf{p} - \mathbf{z}.$$

$$a = |\mathbf{e}|^2, \quad b = 2(\mathbf{f} \cdot \mathbf{e}), \quad c = |\mathbf{f}|^2 - \delta^2, \quad \Delta = b^2 - 4ac.$$

Roots $t = (-b \pm \sqrt{\Delta})/(2a)$ (when $\Delta \geq 0$). Each root with $t \in (0, 1)$ is an intersection; check that the corresponding point lies on the arc.

4.6.2 Arc–arc intersection (isectArcArc)

Two circles of radii δ_A, δ_B centred at $\mathbf{z}_A, \mathbf{z}_B$. Let $d = |\mathbf{z}_B - \mathbf{z}_A|$. The intersection points (up to 2) lie on the radical axis at distance $a_2 = (d^2 + \delta_A^2 - \delta_B^2)/(2d)$ from $\mathbf{z}_A$, with perpendicular offset $h = \sqrt{\max(0, \delta_A^2 - a_2^2)}$:

$$\mathbf{X} = \mathbf{z}_A + a_2 \hat{\mathbf{s}} \pm h \hat{\mathbf{s}}^\perp, \quad \hat{\mathbf{s}} = (\mathbf{z}_B - \mathbf{z}_A)/d.$$

Each candidate is checked against both arc angle ranges.

4.6.3 Edge–edge intersection (isectEdgeEdge)

Lines $\mathbf{p} + t\mathbf{d}$ and $\mathbf{r} + s\mathbf{e}$:

$$t = \frac{(\mathbf{r} - \mathbf{p}) \times \mathbf{e}}{\mathbf{d} \times \mathbf{e}}, \quad s = \frac{(\mathbf{r} - \mathbf{p}) \times \mathbf{d}}{\mathbf{d} \times \mathbf{e}},$$

where $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x$. Intersection if $t, s \in (0, 1)$.

5 Force as Gradient of Intersection Area

The force on vertex $\mathbf{a}_\alpha$ is

$$\mathbf{F}_\alpha = -\frac{\partial E}{\partial \mathbf{a}_\alpha}.$$

We differentiate $E = \text{Area}(\tilde{A} \cap \tilde{B})$ using the Leibniz rule for integrals whose limits and integrands both depend on the parameters.

5.1 General Leibniz rule for signed area

Consider a smooth arc $\gamma(t; \boldsymbol{\theta})$ parameterised by $t \in [t_0(\boldsymbol{\theta}), t_1(\boldsymbol{\theta})]$ where $\boldsymbol{\theta}$ are the polygon vertex coordinates. The signed area contribution is

$$\mathcal{I}(\boldsymbol{\theta}) = \frac{1}{2} \int_{t_0}^{t_1} (\gamma_x \dot{\gamma}_y - \gamma_y \dot{\gamma}_x) dt.$$

Differentiating with respect to $\boldsymbol{\theta}$:

$$\begin{aligned}\frac{\partial \mathcal{I}}{\partial \boldsymbol{\theta}} &= \frac{1}{2} \int_{t_0}^{t_1} \frac{\partial}{\partial \boldsymbol{\theta}} (\gamma_x \dot{\gamma}_y - \gamma_y \dot{\gamma}_x) dt \\ &\quad + \frac{1}{2} (\gamma_x(t_1) \dot{\gamma}_y(t_1) - \gamma_y(t_1) \dot{\gamma}_x(t_1)) \frac{\partial t_1}{\partial \boldsymbol{\theta}} \\ &\quad - \frac{1}{2} (\gamma_x(t_0) \dot{\gamma}_y(t_0) - \gamma_y(t_0) \dot{\gamma}_x(t_0)) \frac{\partial t_0}{\partial \boldsymbol{\theta}}.\end{aligned}\tag{1}$$

The first term is the **direct gradient** (how the curve itself moves with $\boldsymbol{\theta}$, holding limits fixed). The second and third terms are **boundary gradient** contributions: the intersection boundary endpoints move as the polygon vertices change, shifting the limits of integration.

5.2 Gradient for a straight edge segment

For the edge section E_i (from $\mathbf{P} = \mathbf{a}_i^+$ to $\mathbf{Q} = \mathbf{a}_{i+1}^-$) with inside sub-interval $[\mathbf{X}_0, \mathbf{X}_1]$, the energy contribution is $\mathcal{I} = \frac{1}{2}(X_{0x}X_{1y} - X_{0y}X_{1x})$.

Direct term. The segment points are $\boldsymbol{\gamma}(t) = (1-t)\mathbf{P} + t\mathbf{Q}$. The sub-interval endpoints are $\mathbf{X}_0 = \boldsymbol{\gamma}(t_0)$, $\mathbf{X}_1 = \boldsymbol{\gamma}(t_1)$. When $\mathbf{P}$ or $\mathbf{Q}$ moves, all points on the segment move. The force on $\mathbf{P}$ from the direct term is:

$$\left. \frac{\partial \mathcal{I}}{\partial \mathbf{P}} \right|_{\text{direct}} = \frac{\partial}{\partial \mathbf{P}} \left[\frac{1}{2}(X_{0x}X_{1y} - X_{0y}X_{1x}) \right] = \frac{1}{2} \left[(1-t_0) \frac{\partial X_{0x}}{\partial P_x} X_{1y} - \dots \right].$$

More concisely, define $\mathbf{f}_0 = \frac{1}{2}(X_{1y}, -X_{1x})$ and $\mathbf{f}_1 = \frac{1}{2}(-X_{0y}, X_{0x})$. Then:

$$\begin{aligned}\left. \frac{\partial \mathcal{I}}{\partial \mathbf{P}} \right|_{\text{direct}} &= (1-t_0) \mathbf{f}_0 + (1-t_1) \mathbf{f}_1, \\ \left. \frac{\partial \mathcal{I}}{\partial \mathbf{Q}} \right|_{\text{direct}} &= t_0 \mathbf{f}_0 + t_1 \mathbf{f}_1.\end{aligned}$$

Here the factors $(1-t)$ and t are the barycentric weights of $\mathbf{P}$ and $\mathbf{Q}$ in $\boldsymbol{\gamma}(t)$.

Boundary (limit) term. When the endpoints $t_{0,1}$ themselves change (because a crossing parameter depends on $\mathbf{P}$ or $\mathbf{Q}$), we get an additional contribution.

Consider an edge-edge crossing at parameter t_0 : the edge of $\tilde{A}$ (direction $\mathbf{e}_A = \mathbf{Q} - \mathbf{P}$) crosses an edge of $\tilde{B}$ (direction $\mathbf{e}_B$) with $t_0 = (\mathbf{f} \times \mathbf{e}_B) / (\mathbf{e}_A \times \mathbf{e}_B)$. Then:

$$\frac{\partial t_0}{\partial \mathbf{P}} = \frac{-\mathbf{e}_B^\perp + t_0 \mathbf{e}_A^\perp}{\mathbf{e}_A \times \mathbf{e}_B} \quad (2\text{-component vector}).$$

(Here $\mathbf{v}^\perp = (-v_y, v_x)$ is the 90° CCW rotation.) The Leibniz boundary contribution from t_0 is

$$\frac{\partial \mathcal{I}}{\partial t_0} \cdot \frac{\partial t_0}{\partial \mathbf{P}} = \frac{1}{2} (\mathbf{e}_A \times \mathbf{X}_1) \cdot \frac{\partial t_0}{\partial \mathbf{P}},$$

and similarly for t_1 . In the code these are the “ β -correction” terms.

5.3 Gradient for an arc segment

For arc K_i with inside sub-interval $[\tau_0, \tau_1]$, the energy contribution is

$$\mathcal{I} = \frac{1}{2}(X_{0x}X_{1y} - X_{0y}X_{1x}) + \frac{1}{2}\delta^2(\Delta\psi - \sin \Delta\psi), \quad \Delta\psi = (\tau_1 - \tau_0)\Delta\varphi_i.$$

The arc itself $\gamma(\tau) = \mathbf{z}_i + \delta(\cos \phi(\tau), \sin \phi(\tau))$ moves when the backbone vertices move (because $\mathbf{z}_i$, $\phi_{0,i}$, and $\Delta\varphi_i$ all depend on three vertices: $\mathbf{a}_{i-1}$, $\mathbf{a}_i$, $\mathbf{a}_{i+1}$). Only the arc *endpoints* $\tau = 0$ ($\mathbf{a}_i^-$) and $\tau = 1$ ($\mathbf{a}_i^+$) generate simple direct-term gradients (for interior crossings the arc is circular and a separate chain rule through `arcGeomGrad` is needed).

5.4 Arc geometry chain rule (`arcGeomGrad`)

The function `arcGeomGrad` computes, given:

- $\frac{\partial E}{\partial \mathbf{a}_i^-}$ (force on the tangent point from the incoming edge),
- $\frac{\partial E}{\partial \mathbf{a}_i^+}$ (force on the tangent point from the outgoing edge),
- $\frac{\partial E}{\partial \Delta\varphi_i}$ (sensitivity to the arc span),

the resulting forces on backbone vertices $\mathbf{a}_{i-1}$, $\mathbf{a}_i$, $\mathbf{a}_{i+1}$.

The chain rule:

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{a}_{i-1}} &= \frac{\partial E}{\partial \mathbf{a}_i^-} \cdot \frac{\partial \mathbf{a}_i^-}{\partial \mathbf{a}_{i-1}} + \frac{\partial E}{\partial \Delta\varphi_i} \cdot \frac{\partial \Delta\varphi_i}{\partial \mathbf{a}_{i-1}}, \\ \frac{\partial E}{\partial \mathbf{a}_i} &= \frac{\partial E}{\partial \mathbf{a}_i^-} \cdot \frac{\partial \mathbf{a}_i^-}{\partial \mathbf{a}_i} + \frac{\partial E}{\partial \mathbf{a}_i^+} \cdot \frac{\partial \mathbf{a}_i^+}{\partial \mathbf{a}_i} + \frac{\partial E}{\partial \Delta\varphi_i} \cdot \frac{\partial \Delta\varphi_i}{\partial \mathbf{a}_i}, \\ \frac{\partial E}{\partial \mathbf{a}_{i+1}} &= \frac{\partial E}{\partial \mathbf{a}_i^+} \cdot \frac{\partial \mathbf{a}_i^+}{\partial \mathbf{a}_{i+1}} + \frac{\partial E}{\partial \Delta\varphi_i} \cdot \frac{\partial \Delta\varphi_i}{\partial \mathbf{a}_{i+1}}. \end{aligned}$$

Partial derivatives of ℓ_i . $\ell_i = \delta|c_i|/(1 + d_i)$ with $c_i = \hat{\mathbf{u}}_i \times \hat{\mathbf{v}}_i$, $d_i = \hat{\mathbf{u}}_i \cdot \hat{\mathbf{v}}_i$.

Differentiating ℓ with respect to $\hat{\mathbf{u}}$ and $\hat{\mathbf{v}}$:

$$\frac{\partial \ell}{\partial \hat{u}_x} = \frac{1}{1+d}(\text{sgn}(c)\delta\hat{v}_y - \ell\hat{v}_x), \quad \frac{\partial \ell}{\partial \hat{u}_y} = \frac{1}{1+d}(-\text{sgn}(c)\delta\hat{v}_x - \ell\hat{v}_y),$$

and similarly for $\frac{\partial \ell}{\partial \hat{\mathbf{v}}}$ (swap $\hat{\mathbf{u}} \leftrightarrow \hat{\mathbf{v}}$ with sign changes).

Partials of $\Delta\varphi_i$. $\Delta\varphi_i = \text{atan2}(c_i, d_i)$. Using $c^2 + d^2 = 1$:

$$\frac{\partial \Delta\varphi}{\partial \hat{u}_x} = d\hat{v}_y - c\hat{v}_x, \quad \frac{\partial \Delta\varphi}{\partial \hat{u}_y} = -d\hat{v}_x - c\hat{v}_y,$$

and similarly for $\hat{\mathbf{v}}$.

Partials of $\hat{\mathbf{u}}$ with respect to $\mathbf{a}_{i-1}$ and $\mathbf{a}_i$. $\hat{\mathbf{u}} = \mathbf{u}/|\mathbf{u}|$ with $\mathbf{u} = \mathbf{a}_i - \mathbf{a}_{i-1}$.

$$\frac{\partial \hat{u}_j}{\partial a_{i,k}} = \frac{\delta_{jk} - \hat{u}_j \hat{u}_k}{|\mathbf{u}|}, \quad \frac{\partial \hat{u}_j}{\partial a_{i-1,k}} = -\frac{\delta_{jk} - \hat{u}_j \hat{u}_k}{|\mathbf{u}|}.$$

Combining, the code computes (after careful algebraic simplification) the four output force contributions:

$$\mathbf{f}_{i-1} = -\frac{\partial E}{\partial \mathbf{u}}/|\mathbf{u}|, \quad \mathbf{f}_i = \mathbf{f}^{\text{am}} + \mathbf{f}^{\text{ap}} + \frac{\partial E}{\partial \mathbf{u}}/|\mathbf{u}| - \frac{\partial E}{\partial \mathbf{v}}/|\mathbf{v}|, \quad \mathbf{f}_{i+1} = \frac{\partial E}{\partial \mathbf{v}}/|\mathbf{v}|,$$

where $\frac{\partial E}{\partial \mathbf{u}} = (\frac{\partial E}{\partial \hat{\mathbf{u}}} - (\frac{\partial E}{\partial \hat{\mathbf{u}}} \cdot \hat{\mathbf{u}})\hat{\mathbf{u}})/|\mathbf{u}|$ is the force projected perpendicular to $\hat{\mathbf{u}}$ (since $|\hat{\mathbf{u}}| = 1$).

6 SHAKE Constraint Projection

6.1 Constraints

For polygon p with n vertices $\{\mathbf{x}_0, \dots, \mathbf{x}_{n-1}\}$ there are $n+1$ holonomic constraints:

$$C_k(\mathbf{x}) = |\mathbf{x}_{k+1} - \mathbf{x}_k| - L_k = 0, \quad k = 0, \dots, n-1, \quad (2)$$

$$C_n(\mathbf{x}) = A(\mathbf{x}) - A_0 = 0. \quad (3)$$

The constraint Jacobian $J \in \mathbb{R}^{(n+1) \times 2n}$ has rows $\nabla_{\mathbf{x}} C_k$.

6.2 Newton step

SHAKE applies one Newton step per iteration to drive all constraints to zero simultaneously.

The Newton step is: find $\boldsymbol{\lambda} \in \mathbb{R}^{n+1}$ solving

$$(JJ^\top) \boldsymbol{\lambda} = \mathbf{C}(\mathbf{x}),$$

then apply the position update

$$\Delta \mathbf{x} = -J^\top \boldsymbol{\lambda}.$$

6.3 Jacobian structure

Edge constraints. Let $\hat{\mathbf{e}}_k = (\mathbf{x}_{k+1} - \mathbf{x}_k)/|\mathbf{x}_{k+1} - \mathbf{x}_k|$ be the unit edge vector. Then:

$$\nabla_{\mathbf{x}_k} C_k = -\hat{\mathbf{e}}_k, \quad \nabla_{\mathbf{x}_{k+1}} C_k = +\hat{\mathbf{e}}_k, \quad \nabla_{\mathbf{x}_j} C_k = \mathbf{0} \text{ for } j \neq k, k+1.$$

Area constraint. The shoelace formula gives $\nabla_{\mathbf{x}_k} A$. Using $\mathbf{x}_0$ as reference and wrapping all other positions:

$$\begin{aligned} \frac{\partial A}{\partial x_k} &= \frac{1}{2}(y_{k+1} - y_{k-1}), \\ \frac{\partial A}{\partial y_k} &= \frac{1}{2}(x_{k-1} - x_{k+1}). \end{aligned}$$

Denote this gradient vector $\mathbf{g}_k = \nabla_{\mathbf{x}_k} A$.

6.4 Gram matrix $JJ^\top$

The $(n+1) \times (n+1)$ matrix $M = JJ^\top$ has entries $M_{ij} = \nabla C_i \cdot \nabla C_j$.

- **Diagonal, edge-edge:** $M_{kk} = |\nabla_{\mathbf{x}} C_k|^2 = 2$ (the gradient has one $-\hat{\mathbf{e}}_k$ and one $+\hat{\mathbf{e}}_k$, total squared norm 2).
- **Off-diagonal, adjacent edges** ($j = k \pm 1 \pmod n$): The only shared vertex between C_k and C_{k+1} is $\mathbf{x}_{k+1}$.

$$M_{k,k+1} = \nabla C_k|_{\mathbf{x}_{k+1}} \cdot \nabla C_{k+1}|_{\mathbf{x}_{k+1}} = \hat{\mathbf{e}}_k \cdot (-\hat{\mathbf{e}}_{k+1}) = -\hat{\mathbf{e}}_k \cdot \hat{\mathbf{e}}_{k+1}.$$

- **Off-diagonal, non-adjacent edges:** $M_{k,j} = 0$ for $|k - j| > 1 \pmod n$, except for cyclic adjacency).
- **Area row and column:** $M_{n,k} = M_{k,n} = \nabla C_k \cdot \nabla C_n$.

$$M_{n,k} = -\hat{\mathbf{e}}_k \cdot \mathbf{g}_k + \hat{\mathbf{e}}_k \cdot \mathbf{g}_{k+1} = \hat{\mathbf{e}}_k \cdot (\mathbf{g}_{k+1} - \mathbf{g}_k).$$

- **Area diagonal:** $M_{n,n} = |\nabla_{\mathbf{x}} A|^2 = \sum_k |\mathbf{g}_k|^2$.

6.5 LU solve and position update

The $(n+1) \times (n+1)$ system $M\boldsymbol{\lambda} = \mathbf{C}$ is solved by LU factorization (no pivoting, as M is typically well-conditioned for equilateral polygons). Then:

$$\Delta x_k = \lambda_k \hat{e}_{k,x} - \lambda_{k-1} \hat{e}_{k-1,x} - \lambda_n g_{k,x},$$

$$\Delta y_k = \lambda_k \hat{e}_{k,y} - \lambda_{k-1} \hat{e}_{k-1,y} - \lambda_n g_{k,y}.$$

(Thread k handles vertex k ; all threads run in parallel within one block.)

6.6 Convergence

Convergence is checked by $\max_k |C_k(\mathbf{x})| < \text{tol}$ after each iteration, with $\text{tol} = 10^{-15}$. Up to `maxIter` iterations are applied. If `maxIter` is reached, the code retries with $10\times$ more iterations; if it still fails, the FIRE step is rejected (positions reverted).

7 Force Projection

7.1 Goal

After computing the overlap force $\mathbf{F}$, we project out all components that lie in the *constraint subspace* (the span of the constraint gradients). This ensures FIRE moves the system along the constraint manifold, not toward constraint violations.

The projection onto the orthogonal complement of the constraint subspace is:

$$\mathbf{F}_\perp = \mathbf{F} - J^\top (JJ^\top)^{-1} J \mathbf{F},$$

where J is the $(n+1) \times 2n$ constraint Jacobian.

7.2 Edge gradient matrix and SVD

Build G (buildEdgeGradMatrixKernel). For each polygon, construct the $n \times 2n$ matrix G where row k is the gradient of the k -th edge constraint:

$$G_{k,2k} = -\hat{e}_{k,x}, \quad G_{k,2k+1} = -\hat{e}_{k,y}, \quad G_{k,2(k+1)} = +\hat{e}_{k,x}, \quad G_{k,2(k+1)+1} = +\hat{e}_{k,y},$$

all other entries zero.

SVD (cusolverDnDgesvdaStridedBatched). Compute $G = U\Sigma V^\top$ where $U \in \mathbb{R}^{2n \times n}$ and $V \in \mathbb{R}^{n \times n}$. The columns $\mathbf{u}_1, \dots, \mathbf{u}_n$ of U are the left singular vectors and span the range of $G^\top$, i.e. the constraint subspace.

Projection using U . Because $G^\top(GG^\top)^{-1}G = UU^\top$ (when G has full row rank):

$$\mathbf{F}_{\perp, \text{edge}} = \mathbf{F} - UU^\top \mathbf{F} = \mathbf{F} - \sum_{j=1}^n (\mathbf{u}_j \cdot \mathbf{F}) \mathbf{u}_j.$$

7.3 Area gradient orthogonalisation (gramSchmidtAreaKernel)

The area gradient $\mathbf{g}_A = \nabla_{\mathbf{x}} A$ (stored in the `constraints` array) is not generally in the span of the edge gradients. We Gram-Schmidt orthogonalise $\mathbf{g}_A$ against the columns of U :

$$\tilde{\mathbf{g}}_A = \mathbf{g}_A - \sum_{j=1}^n (\mathbf{u}_j \cdot \mathbf{g}_A) \mathbf{u}_j,$$

then normalise:

$$\hat{\mathbf{q}}_A = \tilde{\mathbf{g}}_A / |\tilde{\mathbf{g}}_A|.$$

This is the unit vector in the constraint subspace direction corresponding to the area constraint, orthogonal to all edge constraint directions.

7.4 Full force projection (forceProjectFullKernel)

Project out both edge and area components:

$$\mathbf{F}_{\perp} = \mathbf{F} - \sum_{j=1}^n (\mathbf{u}_j \cdot \mathbf{F}) \mathbf{u}_j - (\hat{\mathbf{q}}_A \cdot \mathbf{F}) \hat{\mathbf{q}}_A.$$

The result $\mathbf{F}_{\perp}$ has no component in the direction of any constraint gradient, so FIRE will not drive constraint violations.

8 FIRE Minimization

8.1 Background

FIRE (Fast Inertial Relaxation Engine, Bitzek *et al.* 2006) is a molecular dynamics-based minimizer that uses an inertial term to accelerate convergence toward energy minima. The key idea is to run a modified velocity-Verlet integrator and periodically align the velocity vector with the force direction.

8.2 Velocity-Verlet integration

For a particle at position $\mathbf{x}$ with velocity $\mathbf{v}$ and force $\mathbf{F}$:

1. **Half-velocity update** (`updatePositionAndVelocityFIREKernel`, first half):

$$\mathbf{v}^{n+1/2} = \mathbf{v}^n + \frac{1}{2}\mathbf{F}^n dt,$$

2. **Position update** (same kernel):

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \mathbf{v}^{n+1/2} dt + \frac{1}{2}\mathbf{F}^n (dt)^2 = \mathbf{x}^n + \mathbf{v}^n dt + \frac{1}{2}\mathbf{F}^n (dt)^2,$$

followed by periodic wrap $x \mapsto x - \lfloor x \rfloor$.

3. **SHAKE**: project $\mathbf{x}^{n+1}$ onto the constraint manifold.
4. **Velocity rederivation** (`rederiveVelocityFromDisplacementKernel`): after SHAKE may move positions significantly, the velocity is rederived from the actual displacement:

$$\mathbf{v}_{\text{redep}}^{n+1/2} = \text{wrap}(\mathbf{x}_{\text{SHAKE}}^{n+1} - \mathbf{x}^n)/dt.$$

5. **Force and projection**: compute $\mathbf{F}^{n+1}$ via `updateForceEnergyRounded` + `projectForce`.
6. **Second half-velocity update** (`updateVelocityFIREKernel`):

$$\mathbf{v}^{n+1} = \mathbf{v}_{\text{redep}}^{n+1/2} + \frac{1}{2}\mathbf{F}^{n+1} dt.$$

8.3 FIRE steering

After each step, compute the *power*:

$$P = \mathbf{F}^{n+1} \cdot \mathbf{v}^{n+1}.$$

If $P > 0$ (moving toward lower energy):

1. Bend velocity toward force direction:

$$\mathbf{v} \leftarrow (1 - \alpha) \mathbf{v} + \alpha \frac{|\mathbf{v}|}{|\mathbf{F}|} \mathbf{F}.$$

2. Increment n_+ (the count of consecutive positive-power steps). If $n_+ \geq n_{\text{min}}$, increase dt :

$$dt \leftarrow \min(dt \cdot f_{\text{inc}}, dt_{\text{max}}), \quad \alpha \leftarrow \alpha \cdot f_{\alpha}.$$

If $P \leq 0$ (moving uphill):

1. Zero the velocity: $\mathbf{v} \leftarrow \mathbf{0}$.
2. Decrease dt :

$$dt \leftarrow dt \cdot f_{\text{dec}}.$$

3. Reset $\alpha \leftarrow \alpha_{\text{start}}$, $n_+ \leftarrow 0$.

Parameters (defaults used in the packing test):

Parameter	Symbol	Value
Initial time step	dt_0	10^{-3}
Maximum time step	dt_{max}	5×10^{-2}
Initial velocity mixing	α_{start}	0.1
Mixing decay	f_{α}	0.99
dt increase factor	f_{inc}	1.1
dt decrease factor	f_{dec}	0.5
Min steps before increase	n_{min}	5

8.4 Time-step scaling for dense packings

For a system with maximum force F_{max} and minimum edge length L_{min} , a safe time step is determined by requiring that the maximum displacement per step at dt_{max} remains small compared to L_{min} :

$$v_{\text{max}} dt_{\text{max}} \lesssim \varepsilon L_{\text{min}}, \quad v_{\text{max}} \sim F_{\text{max}} dt_{\text{max}}.$$

This gives

$$dt_{\text{max}} \lesssim \sqrt{\varepsilon L_{\text{min}} / F_{\text{max}}}.$$

For $\varepsilon = 10^{-3}$, $L_{\text{min}} \approx EL_{\text{small}}$, and F_{max} from the initial configuration.

9 Shape Generation via SLSQP

9.1 Goal

Generate a random equilateral closed polygon with N sides, shape index κ , and backbone area A_{tgt} , by optimizing a random distribution of interior angles subject to closure, equilateral, and area constraints.

9.2 Angle parameterisation

Parameterise the polygon by its $N - 1$ free interior turning angles $\boldsymbol{\phi} = (\phi_0, \dots, \phi_{N-2}) \in (0, \pi)^{N-1}$ (the N -th angle is fixed by the closure condition $\sum_k \phi_k = 2\pi$ minus the last).

Given $\boldsymbol{\phi}$, construct the vertices:

$$\theta_k = \sum_{j=0}^{k-1} \phi_j, \quad \hat{\mathbf{e}}_k = (\cos \theta_k, \sin \theta_k), \quad \mathbf{v}_0 = \mathbf{0}, \quad \mathbf{v}_{k+1} = \mathbf{v}_k + \hat{\mathbf{e}}_k.$$

(Unit edge lengths; all $L_k = 1$.)

9.3 Closure constraint

For the polygon to close: $\mathbf{v}_N = \mathbf{v}_0 = \mathbf{0}$, i.e.:

$$\sum_{k=0}^{N-1} \hat{\mathbf{e}}_k = \mathbf{0}.$$

This gives 2 equality constraints on ϕ .

9.4 Shape index constraint

The polygon has all edges of unit length, so perimeter $P = N$. The area is $A = A(\phi)$. The shape index is $\kappa = P/\sqrt{A} = N/\sqrt{A}$. Requiring $\kappa = \kappa_0$ is equivalent to requiring

$$A(\phi) = (N/\kappa_0)^2 =: A_{\text{unit}}.$$

This is 1 additional equality constraint.

9.5 SLSQP formulation

$$\begin{aligned} \min_{\phi} \quad & \frac{1}{2} |\phi - \phi_0|^2 \quad (\text{stay close to random reference}) \\ \text{s.t.} \quad & \sum_k \cos \theta_k(\phi) = 0, \\ & \sum_k \sin \theta_k(\phi) = 0, \\ & A(\phi) = A_{\text{unit}}, \\ & \pi > \sum_k \phi_k > \pi, \quad \phi_k \in (10^{-4}, \pi - 10^{-4}). \end{aligned}$$

(The range constraint on $\sum \phi_k$ is a convenience to avoid self-intersections and ensure a convex-ish polygon; the bounds on ϕ_k prevent degenerate angles.)

The reference ϕ_0 is drawn from a Dirichlet distribution scaled to $[0, 2\pi]$, giving random convex-ish shapes. If SLSQP succeeds, the resulting unit-area polygon is scaled to area A_{tgt} :

$$\mathbf{v}_k \leftarrow \sqrt{A_{\text{tgt}}/A_{\text{unit}}} (\mathbf{v}_k - \bar{\mathbf{v}}),$$

where $\bar{\mathbf{v}}$ is the centre of mass. A random rotation is applied to break any orientation bias.

10 Bidisperse Packing Setup

10.1 Target areas

The packing has $N_L = N_S = N_{\text{poly}}/2$ large and small polygons in a unit $[0, 1)^2$ box. The *backbone packing fraction* is:

$$\phi = N_L A_L + N_S A_S.$$

The large and small polygons have the same shape index κ but the large polygon's perimeter is ratio times larger than the small polygon's:

$$P_L = \text{ratio} \cdot P_S \implies A_L = \text{ratio}^2 \cdot A_S.$$

Substituting into the packing fraction:

$$\phi = N_L \text{ratio}^2 A_S + N_S A_S = A_S (N_L \text{ratio}^2 + N_S),$$

$$A_S = \frac{\phi}{N_L \text{ratio}^2 + N_S}, \quad A_L = \text{ratio}^2 A_S.$$

10.2 Target edge lengths

For a polygon with shape index κ , backbone area A , and N equilateral edges of length L :

$$P = NL = \kappa \sqrt{A} \implies L = \frac{\kappa \sqrt{A}}{N}.$$

SHAKE uses these target edge lengths to maintain equilateral polygons.

10.3 Rounding radius

The rounding radius δ is chosen relative to the *smaller* polygon's edge length:

$$\delta = 0.15 L_S = \frac{0.15 \kappa \sqrt{A_S}}{N}.$$

This ensures the validity condition $\delta < L/(2 \tan(\pi/N)) \approx L \cdot N/(2\pi)$ (for large N , $\tan(\pi/N) \approx \pi/N$) is comfortably satisfied for both polygon sizes.

10.4 Packing fraction for the 2-polygon test

For the two-polygon test we use a reference large polygon area of $A_L = 0.8/64$ (same as the per-polygon area in a 64-polygon packing at $\phi = 0.8$). The small polygon is obtained by $A_S = A_L/\text{ratio}^2$. In this case:

$$A_L = \frac{0.8}{64} = 0.01250, \quad A_S = \frac{0.01250}{1.96} \approx 0.006378.$$

11 Periodic Boundary Conditions

All positions live in $[0, 1)^2$. The minimum-image convention is:

$$\text{wrap}(x) = x - \lfloor x + \frac{1}{2} \rfloor.$$

When computing edge vectors $\mathbf{u} = \mathbf{a}_k - \mathbf{a}_{k-1}$, the wrap function is applied component-wise to $a_{k,x} - a_{k-1,x}$ and $a_{k,y} - a_{k-1,y}$.

For computing the interaction between polygon A (where vertex i lives) and polygon B , all of B 's vertices are placed in the same image as B 's first vertex, which is placed at the minimum-image position relative to vertex i :

$$\mathbf{b}_0^{\text{MIC}} = \mathbf{a}_i + \text{wrap}(\mathbf{b}_0 - \mathbf{a}_i), \quad \mathbf{b}_j^{\text{MIC}} = \mathbf{b}_0^{\text{MIC}} + (\mathbf{b}_j - \mathbf{b}_0).$$

This ensures the entire polygon B is in a single consistent periodic image, preventing artefacts from vertices on opposite sides of the wrap boundary.

12 Summary of Computational Flow

1. **Initialization:** Generate N_{poly} random equilateral polygons with target areas and shape index via SLSQP. Place on jittered grid. Initialize neighbour cells.
2. **Force/energy evaluation** (`updateForceEnergyRoundedCUDA`): For each vertex i of polygon A , in parallel:
 - (a) Compute arc geometry at i and `next[i]`.
 - (b) Find all polygons $B \neq A$ in the 3×3 neighbourhood of i 's cell.
 - (c) For arc K_i : find crossings with all edges/arcs of $\tilde{B}$; integrate signed area over inside sub-intervals.
 - (d) For edge E_i : similarly.
 - (e) Accumulate forces on $\{i-1, i, i+1, i+2\}$ via chain rule through arc geometry; also update forces on B 's vertices.
3. **Force projection** (`projectForceCUDA`): SVD of edge-gradient matrix + Gram-Schmidt area projection.
4. **FIRE step:** Half-velocity update $\rightarrow$ position update $\rightarrow$ SHAKE $\rightarrow$ velocity rederivation $\rightarrow$ force evaluation $\rightarrow$ second half-velocity $\rightarrow$ FIRE steering.
5. Repeat until $|\mathbf{F}_\perp|_\infty < F_{\text{tol}}$ or maximum steps.